

关于若干问题的研讨

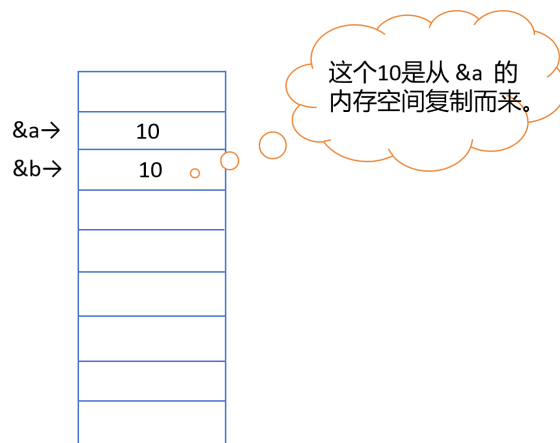
- 有同学问我关于函数值返回的问题，像该名同学这样对函数返回存在认识不全的情况可能是普遍的，所以写一篇文档，供大家学习参考之用。
- 有同学在QQ联系我，问是否要仔细学习C++的语法，因为她听不懂。我一问才知道，原来她不是不懂C++，而是C没学懂。所以建议她认真学习C，连基本语法都不懂，那就不谈后面的编程的。

一、数据

1.普通数据

```
1 int main(){
2     int a = 10;
3     int b = a;
4     return 0;
5 }
```

像上面的代码，相信大家都能明白，但我仍画个图。



由于 `&a`和`&b`指向的是两块内存空间，所以

```
1 a = 11;
2 cout<<b; // 输出仍然是10.
```

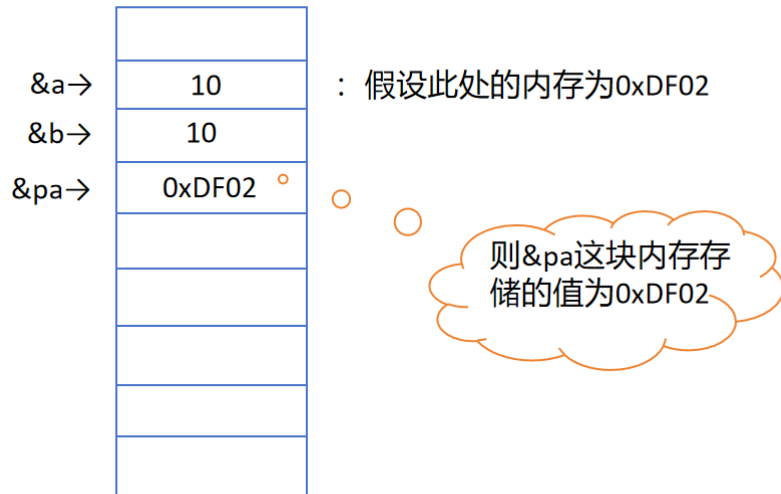
2.指针

指针的本质是，存储的不是普通数字，而是内存地址。这里，我就不用传统的“指向”二字来解释，而是通过数值来讲述。

```

1  int main(){
2      int a = 10;
3      int b = a;
4      int *pa = &a;
5      return 0;
6  }

```

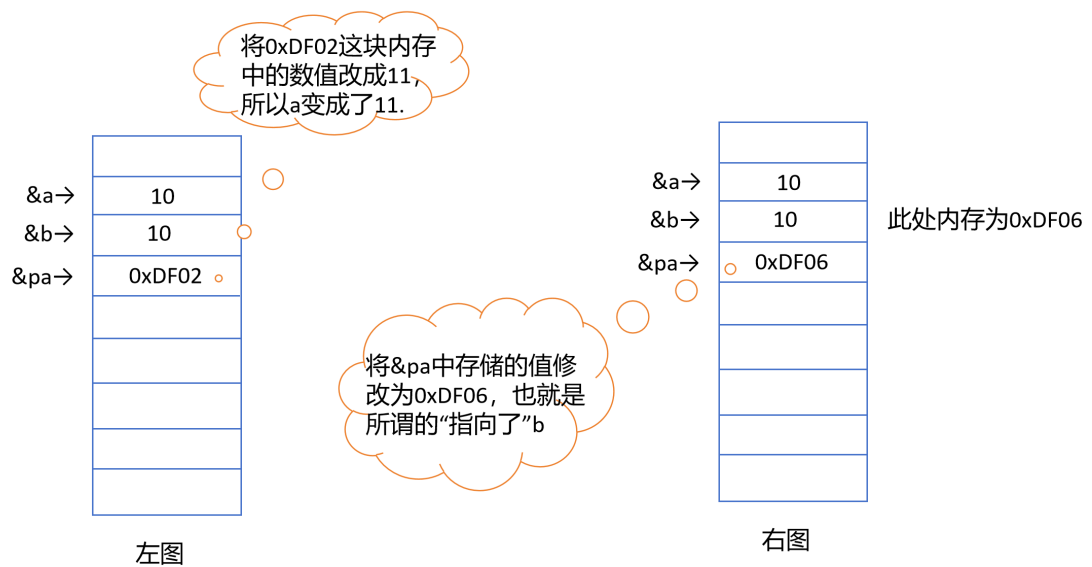


当指针数值发生改变时，情况分为多种。如下：

```

1  int main(){
2      int a = 10;
3      int b = a;
4      int *pa = &a;
5
6      /**数值改变*/
7      *pa = 11; // 见下图（左图）
8      pa = &b; // 见下图（右图）
9      return 0;
10 }

```



一个指针变量，它存储的值能改变，但也是一个明确的地址，否则会报错。如下：

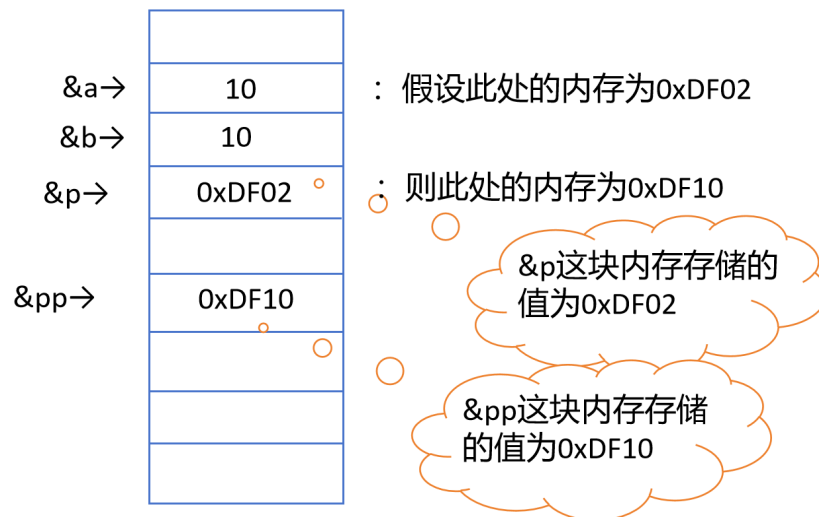
```
1 | pa = 11; // 指针不能直接赋值
```

3.二级指针

本不应该来说二级指针，因为一级都令人困惑。但是，通过二级指针，可以帮助大伙理解指针。

```
1 | int main(){  
2 |     int a = 10;  
3 |     int b = a;  
4 |     int *p = &a;  
5 |     int **pp = &p;  
6 |     return 0;  
7 | }
```

内存构造如下：



看看以下代码哪些是正确的？

```

1      p = 10; // 错误，指针不能直接赋值
2      *p = 11; // 正确，
3      p = &b; // 正确
4
5
6      pp = 10; // 错误，指针不能直接赋值
7      pp = p; // 错误，虽然p里面存储的也是一个地址，但它是一级的，不是二级的。
8              // 所谓一级吗，即*p能直接赋值了。
9              // 如果 pp = p正确，则意味着，*pp也能被直接赋值，但实际上是不能的。
10             // 因此，*pp也必须是一个地址，且是一级地址。
11      *pp = &b; // 正确，pp是二级指针，*pp就是一级指针，&b也是一级地址。
  
```

二、数值的复制

数值的复制，我们不需要举那么多例子，主要讲解函数的返回。

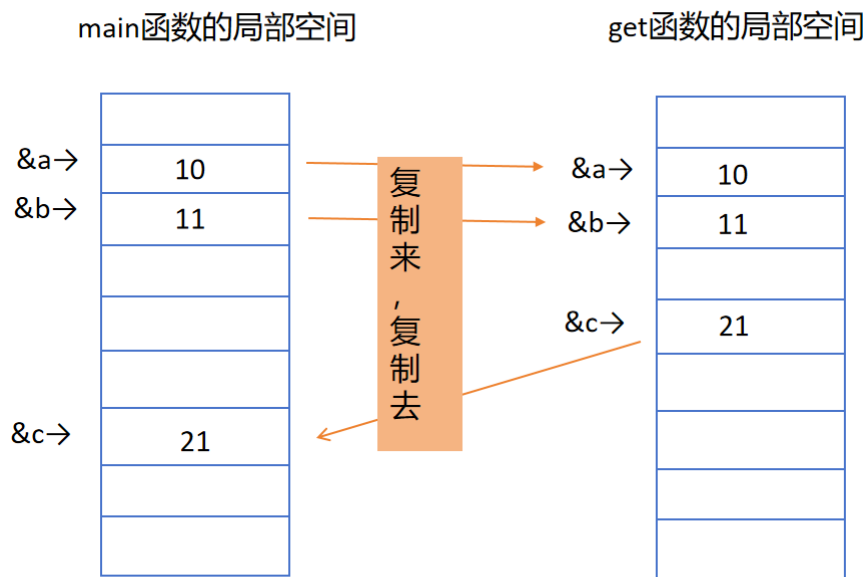
首先，我们要明确，一个函数在运行结束后，它所占据的局部数据空间是被释放掉了——也就是不存在了。

1.普通的数值

```

1  int get(int a,int b){
2      int c = a + b;
3      return c;
4  }
5
6  int main(){
7      int a = 10;
8      int b = 11;
9      int c = get(a,b);
10     return 0;
11 }

```



每个数值都有自己的存储空间，它们之间是通过复制来实现的。

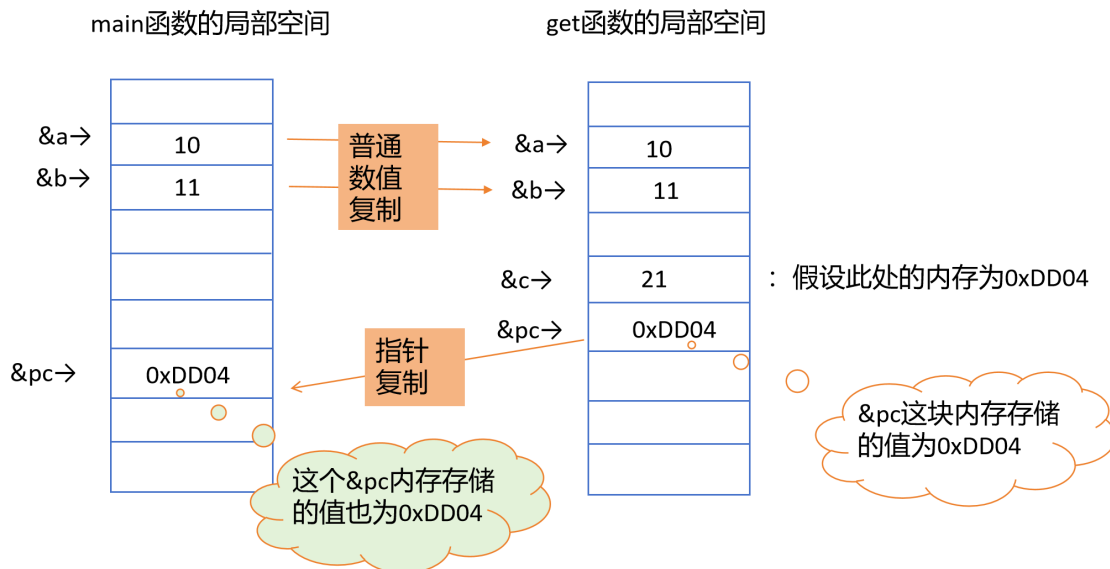
2. 指针传递

我们将上一个编程改一下，问题就出来了。

```

1  int* get(int a,int b){
2      int c = a + b;
3      int *pc = &c;
4      return pc ;
5  }
6
7  int main(){
8      int a = 10;
9      int b = 11;
10     int *pc = get(a,b); //隐形的BUG
11     return 0;
12 }

```



当`get`函数运行结束，`&c`这块内存没了，释放了或者被覆盖了，总而言之，`&c`里面存储的可能不再是21，而是一个狗屎值。

3.用malloc解决的问题

我们在课堂上一再强调用`malloc`来解决函数中定义的指针问题。

```

1  int* get(){
2      int c = 0;
3      int *p = &c;
4      return p;
5  }
6  int main(){
7      int *p = get(); // 出问题，因为当get运行结束，&c也被释放
8      return 0;
9  }

```

修改：

```

1  int* get(){
2      int c = 0;
3      int *p = (int) malloc(sizeof(int));
4      *p = c;
5      return p;
6  }
7  int main(){
8      int *p = get(); // 没有问题，
9      return 0;
10 }

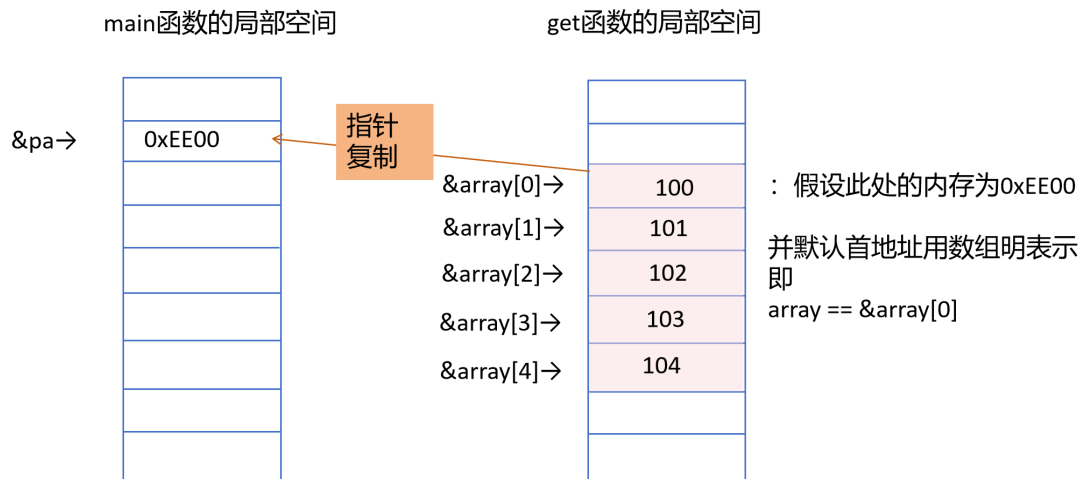
```

4.数组的赋值

```

1  #define MAX 5
2  int* get(){
3      int array[MAX] = {100,101,102,103,104};
4      return a;
5  }
6  int main(){
7      int *pa = get();
8      return 0;
9  }

```



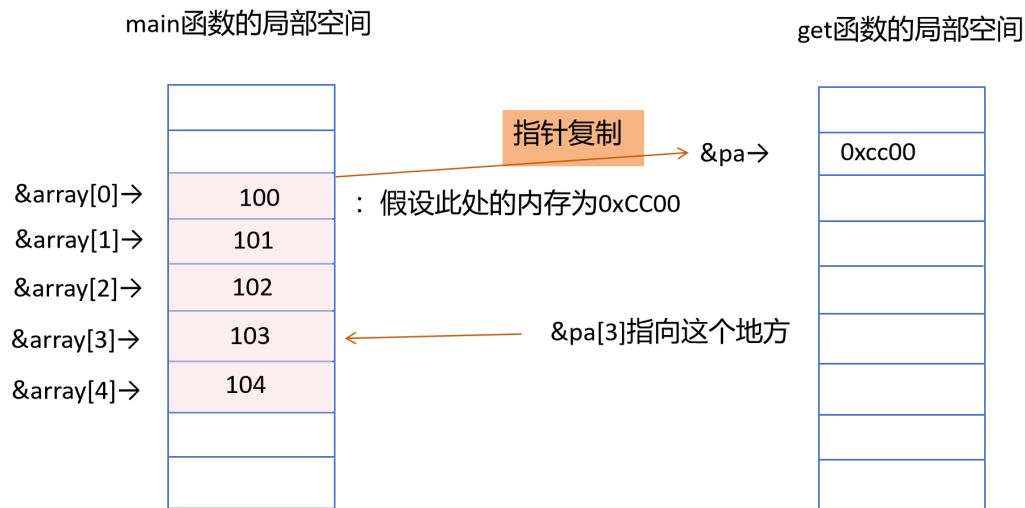
表面上，pa可以操作0xEE00这个内存区域，但是，当get函数结束，这里面的一切都没了。pa指向的这个空间什么值？不知道！！！也就成了野指针。

但是这样做就没问题了。

```

1  #define MAX 5
2  void get(int pa[]){
3      pa[3] += 10;
4  }
5  int main(){
6      int array[MAX] = {100,101,102,103,104};
7      get();
8      cout<<array[3] // 变成了113
9      return 0;
10 }

```



总结一下：其实就是看内存空间有没有被释放。main函数的空间也会被释放，但是那时候，整个程序都结束了，也就无所谓了。

三、我为什么要引入到C++?

将C引入到C++本身就课程计划的一部分。——不是临时起意的。但是我们不需要从零学C++的语法，只需知道以下几点。

1.基本语法

```
1 //以前c语言的头
2 #include<stdio.h>
3
4
5 //现在换成
6 #include<iostream>
7 //还要加上
8 using namespace std;
```

```
1 //以前的
2 printf("%d",n);
3 scanf("%d",&n);
4
5 //现在的
6 cout<<n<<endl; //endl是换行的意思。end line
7 cin >> n; // 这里不用取地址。
```

```
1 //以前的
2 #include<string.h>
3 int main(){
4     char*p = "123";
```



```

5     return 0;
6 }
7
8
9
10 //现在的
11 #include<string>
12 int main(){
13     string a = "123";
14     string b = "abc";
15     string c = a+b; // 123abc
16     return 0;
17 }

```

2.什么是对象？

什么是对象？

说到对象，大家不要慌。class 是类，由类生成出来的就是对象。如下：

```

1  /**这是一个类*/
2  class Person{
3      public:
4          int id;
5          char *name;
6  };
7
8  int main(){
9      Person wzd; // 这就是一个对象
10     return 0;
11 }

```

如果大家对class 理解不透彻，那就把它想象成struct——本质上没什么区别。

如果大家对struct理解不透彻，那就将struct理解成和int、float、double、char 一样的东西，它是自定义数据类型嘛！！

3.主要使用STL

STL是一个很伟大的库，它解决新手99%的问题。我们需要记住。

```
1 #include <vector>
2 using namespace std;
3
4 int main(){
5     vector<string> v;// 一个用来存储字符串的数组，我们以前用的是 char**
6
7     v.push_back("wzd01");
8     v.push_back("wzd02");
9     /**你们自己说，谁方便??? */
10    return 0;
11 }
12
```

```
1 #include <stack>
2 using namespace std;
3
4 int main(){
5     stack<char> s;// 一个栈
6
7     s.push('(');
8     cout<<s.top();
9     s.pop();
10    return 0;
11 }
12
```